Forum

LB PROJEKT

Nadine Duc | Modul 223 | 21.03.2026
https://github.com/WISS-Student/Modul_223_Projekt

Inhaltsverzeichnis

Projektidee 2
User Stories 3
User Story 1: 3
User Story 2: 3
User Story 3: 3
User Story 4: 3
Backend 4
Frontend 5
Darstellung 6
Sicherheitskonzept 7
Hilfestellung 8

Projektidee

Das Ziel dieses Projekts ist die Entwicklung einer funktionaler Web-Plattform in der Gestalt von einem Forum, auf welcher sich verschiedene Benutzer über ihre Erfahrungen austauschen können. Die Applikation ermöglicht es Nutzern, eigene Beiträge zu verfassen, Bilder zu teilen und die Inhalte anderer Mitglieder zu kommentieren.

Warum im Stil von einem Forum?

In meiner Kindheit war ich sehr oft auf dem Netz. Forum Seiten waren damals noch die Haupt „Social Networks“, auf denen beobachtete ich Leute, wie sie sich über Spiele, Kunst oder Anime austauschten.

Ich liebe das Konzept von einer Thematisierten Seite. Ausserdem sehen Forum Layouts auch sehr schön aus.

Was war das wichtigste was diese Web-Applikation erreichen sollte?

- Man soll einen User erstellen können und sich damit auch einloggen können.
- Man soll Posts erstellen und editieren können und bei denen auch optional Bilder anhängen können.
- Man soll auf Posts kommentieren können und diese Kommentare zeigen die Benutzer und das Datum an.
- Schlussendlich soll ein „Admin“ User Posts entfernen können, für Fälle wo sich ein Nutzer z.B sehr feindlich gegenüber anderen verhält oder gegen die Regeln verstösst.

USER STORIES

User Story 1:

"Als nicht angemeldeter Besucher möchte ich mir die Forum posts anschauen, um etwas über dieses Thema zu lernen"

+ Kann die Seite anschauen

+ Kann Posts & Kommentare lesen

User Story 2:

"Als nicht angemeldeter Besucher möchte ich einen Account erstellen, um an dem Forum teilzunehmen"

Gleiche Permissions wie nicht angemeldete User & angemeldete User

+ Kann sich einloggen

+ Hat Account

User Story 3:

"Als User auf der Seite möchte ich über meine Erfahrungen Posten inklusive meinen Bildern, um anderen Usern meine Kreationen zu zeigen"

Gleiche Permissions wie nicht angemeldete User

+ Posts veröffentlichen

+ Eigene Posts editieren können

+ Eigene Posts entfernen können

+ Posts können Bilder Anhänge haben

User Story 4:

"Als User auf der Seite möchte ich auf anderen Posts kommentieren können, um mich mit anderen Usern auszutauschen"

Gleiche Permissions wie nicht angemeldete User & angemeldete User

+ Auf Posts kommentieren können

BACKEND

Das Backend gilt als das Grundgerüst für das Projekt. Es regelt alle Funktionen, Erlaubnisse und die Persistenz der Daten.

Controller**:** Definieren die REST-Endpunkte für die Kommunikation mit dem Frontend (z.B. /api/posts, /api/comments).

Services**:** Hier ist die zentrale Geschäftslogik gekapselt, wie die Validierung von Nutzerrechten vor Löschvorgängen. Das bedeutet die Überprüfung ob der Post dem Nutzer gehört und dadurch dem Postinhaber es ermöglicht diesen zu Löschen oder zu editieren.

Repositories**:** Nutzen Spring Data JPA für einen objektrelationalen Zugriff auf die MySQL Datenbank.

Models**:** Die Datenstrukturen wie User, Post und Comment mit ihren Beziehungen (ManyToOne)

Datenhaltung & Persistenz

Technologie
Die dauerhafte Speicherung aller Anwendungsdaten erfolgt in einer MySQL Datenbank. In dieser Datenbank werden Daten, wie der Benutzer, Posts, Kommentare und Rollen abgespeichert. Die Anbindung an die Java-Anwendung wird über Spring Data JPA realisiert, was eine effiziente Abstraktion der SQL-Abfragen ermöglicht. In anderen Applikationen könnte man auch Hibernate verwenden welche dann auf Datenbanken mit „HQL“ verlinkt, hier benutze ich jedoch die JDBC.

Automatisierung
Mit unserem Springboot Initialiser wird es ermöglicht die Hibernate-Eigenschaft spring.jpa.hibernate.ddl-auto=update zu verwenden. Diese sorgt dafür, dass das Datenbankschema automatisch mit den Java-Entities synchronisiert wird.

Initialisierung
Der CommandLineRunner in der Main-Klasse des Backends sorgt dafür, dass alle nötigen Daten erstellt werden beim Aufstart. Dieser prüft beim Anwendungsstart, ob die für das Berechtigungskonzept essenziellen Rollen (ROLE_USER, ROLE_ADMIN) bereits in der Datenbank existieren. Falls nicht, werden diese automatisch angelegt.

Testing

Backend Testing ist integriert in PostControllerTest.java.

Bei den ersten zwei Versuchen versagte der erste Test wegen 403 Unauthorized Errors. Dies konnte ich beheben mit einem Filter, welcher die Rollen konfiguration ignoriert.

Der Zweite Test ging ohne Probleme durch und wirft wie erwartet einen Error, wenn man einen Post sucht der nicht existiert.

FRONTEND

Das Frontend ist eine dynamische Single-Page-Application (SPA), die mit React aufgebaut wurde. Sie ermöglicht es alle nötigen Funktionen aus dem Backend auch auf der Web-Applikation anzuzeigen und deren Nutzen zu verbinden. Ausserdem kann man mit den CSS Dateien die Darstellung verschönern.

Routing: React Router ermöglicht eine nahtlose Navigation zwischen Dashboard, Login und Beitragsansichten ohne Seiten-Reloads.

State Management: Die Context API (AuthContext) verwaltet den globalen Authentifizierungsstatus, sodass Benutzerinformationen überall in der App verfügbar sind.

API-Kommunikation: Der Axios Import (api.js) sorgt dafür, dass der JWT-Token automatisch bei jedem Request im Authorization-Header mitgesendet wird.

Modularität: Die Trennung in funktionale Module z.B. PostList & CreatePost und wiederverwendbare UI-Komponenten z.B. PostCard & Sidebar garantiert eine hohe Wartbarkeit. Das habe ich in meinem Geschäft gelernt welches bis zu 4000 verschiedene Controller Klassen und Page Fragments benützt.

Testing

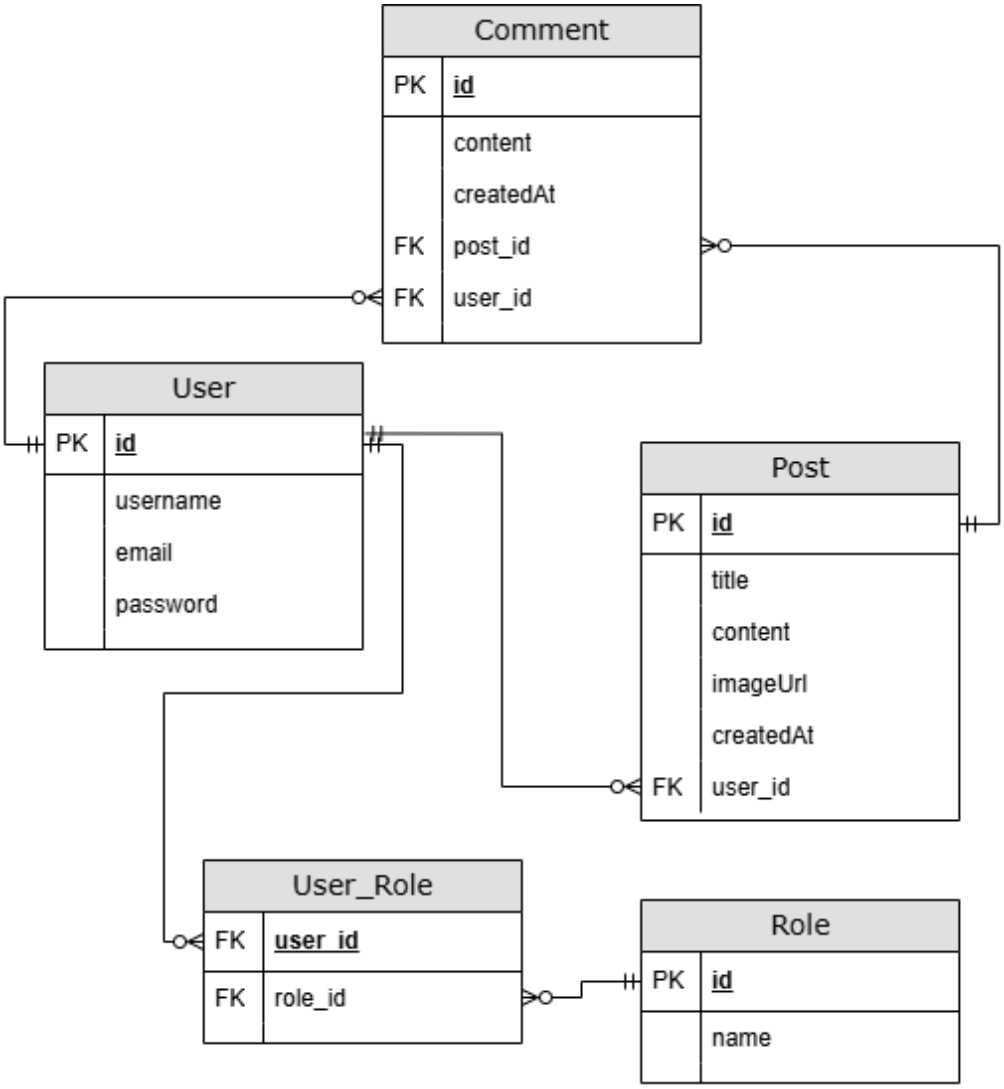
Auch im Frontend habe ich zwei Tests integriert, eines im AuthGuard.test.jsx und das andere im PostCard.test.jsx.

Der erste Test soll schauen ob ein „Guest“ User Posts erstellen kann. Die eigentliche Funktion sollte ein redirect auf die Login Seite sein. Bei der Ansicht im Projekt erscheint zwar der Create Post Button gar nicht, würde es jemand allerdings trotzdem schaffen die Create Post Page aufzurufen, dann sorgt PrivateRoute dafür den Guest umzuleiten.

Der zweite Test ist relativ simpel, es soll einfach überprüfen ob den Post den man erstellt hat, danach auch anschauen kann mit allen Inhalten richtig vorhanden.

Diese Tests liefen beide gut und kamen nur einmal mit einem Error zurück, welches daran lag, dass ich kein „vitest.config.js“ hatte.

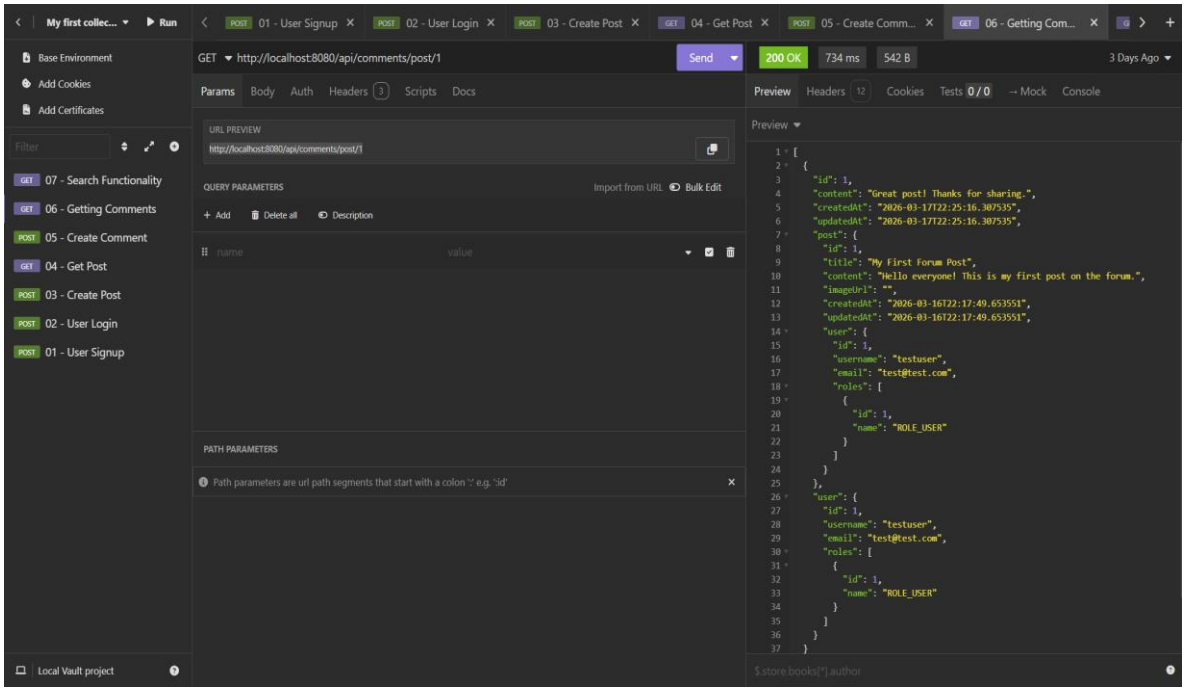
DARSTELLUNG



SICHERHEITSKONZEPT

Das Sicherheitskonzept in meinem Projekt ist End-to-End Sicherheit & Datenschutz.

- JWT-Authentifizierung: Nach dem Login erhält der User einen verschlüsselten Token, der lokal gespeichert wird. Dies ermöglicht eine zustandslose und sichere Kommunikation.
- RBAC (Role-Based Access Control): Durch Spring Security wird sichergestellt, dass sensible Aktionen (wie das Erstellen von Posts) nur für authentifizierte Nutzer mit der entsprechenden Rolle freigeschaltet sind.
- Integrität: Passwörter werden mittels BCrypt gehasht, sodass selbst bei einem Datenbank-Zugriff keine Klartext-Passwörter ausgelesen werden können.



HILFESTELLUNG

Inspiration für das Forum Projekt

https://www.reddit.com/

Inspiration für die Darstellung

https://www.elitefourum.com/c/general-board/9

Hilfe mit dem Code

Aufgaben Im Unterricht -> Mein Grundgerüst

Sven Schirmer

Webseiten

GeeksforGeeks **https://www.geeksforgeeks.org/**

Spring Boot – Annotations & Spring Boot – JWT Authentication

Spring Data JPA – Repository Interface

Spring Security – Role Based Access Control

Spring – @Autowired Annotation

React Router - Programmatic Navigation & URL Params

React Context API and useContext Hook

How to use Axios Interceptors in ReactJS?

ReactJS - Handling Forms

Baeldung **https://www.baeldung.com/**

Spring @RestController vs @Controller

Spring Security with JWT

Introduction to Spring Data JPA & Introduction to Spring Method Security

Guide to Spring @Autowired

React Router Hooks: useNavigate and useParams

The React Context API

Guide to Axios Interceptors

Forms in React

Troubleshooting mit AI

https://lumo.proton.me/

Tools

Spring Initializr	https://start.spring.io/
React	https://react.dev/
Insomnia	https://insomnia.rest/
VSCode	https://code.visualstudio.com/